



Acunetix Threat Level 4

One or more critical-severity type vulnerabilities have been discovered by the scanner. A malicious user can exploit these vulnerabilities and compromise the backend database and/or deface your website.

Scan Detail

Target	http://192.168.0.103
Scan Type	Full Scan
Start Time	Apr 19, 2026, 10:22:08 AM GMT
Scan Duration	29 minutes
Requests	128665
Average Response Time	14ms
Maximum Response Time	29637ms
Application Build	v24.6.240626115
Authentication Profile	-

13

Critical

39

High

64

Medium

11

Low

5

Informational

Severity	Vulnerabilities	Instances
 Critical	13	13
 High	39	39
 Medium	64	64
 Low	11	11
 Informational	5	5
Total	132	132

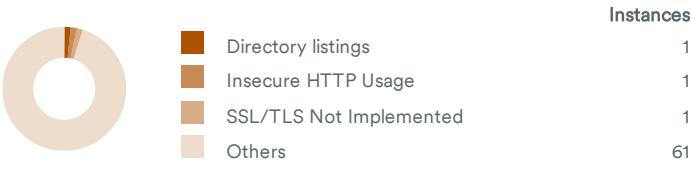
Critical Severity



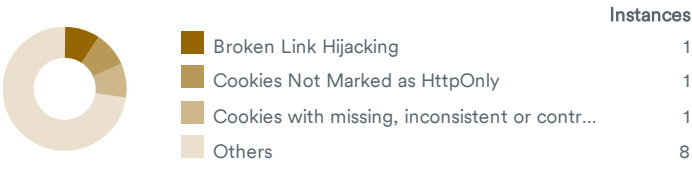
High Severity



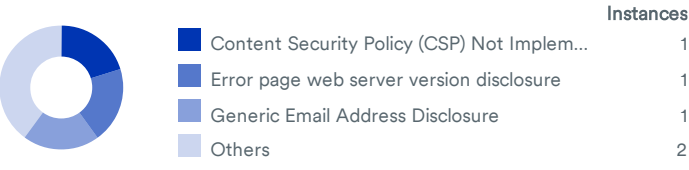
Medium Severity



Low Severity



Informational



Impacts

SEVERITY	IMPACT
 Critical	 WordPress CVE-2020-28039 Vulnerability
 Critical	 WordPress Deserialization of Untrusted Data Vulnerability
 Critical	 WordPress Deserialization of Untrusted Data Vulnerability
 Critical	 WordPress Deserialization of Untrusted Data Vulnerability
 Critical	 WordPress Improper Input Validation Vulnerability
 Critical	 WordPress Improper Input Validation Vulnerability
 Critical	 WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability
 Critical	 WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability
 Critical	 WordPress Improper Privilege Management Vulnerability
 Critical	 WordPress Improper Privilege Management Vulnerability
 Critical	 WordPress Other Vulnerability
 Critical	 WordPress Server-Side Request Forgery (SSRF) Vulnerability
 Critical	 WordPress Server-Side Request Forgery (SSRF) Vulnerability
 High	 WordPress 4.7.x Arbitrary File Deletion Vulnerability
 High	 WordPress 4.7.x Cross-Site Request Forgery
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x Multiple Vulnerabilities
 High	 WordPress 4.7.x PHP Object Injection
 High	 WordPress 4.7.x Prototype Pollution
 High	 WordPress Access of Resource Using Incompatible Type ('Type Confusion') Vulnerability

SEVERITY	IMPACT
 High	1 WordPress Cross-Site Request Forgery (CSRF) Vulnerability
 High	1 WordPress CVE-2019-17673 Vulnerability
 High	1 WordPress CVE-2020-28033 Vulnerability
 High	1 WordPress Deserialization of Untrusted Data Vulnerability
 High	1 WordPress Deserialization of Untrusted Data Vulnerability
 High	1 WordPress Directory Traversal
 High	1 WordPress Exposure of Sensitive Information to an Unauthorized Actor Vulnerability
 High	1 WordPress Improper Control of Generation of Code ('Code Injection') Vulnerability
 High	1 WordPress Improper Input Validation Vulnerability
 High	1 WordPress Improper Input Validation Vulnerability
 High	1 WordPress Improper Input Validation Vulnerability
 High	1 WordPress Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') Vulnerability
 High	1 WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability
 High	1 WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability
 High	1 WordPress Inadequate Encryption Strength Vulnerability
 High	1 WordPress Missing Authentication for Critical Function Vulnerability
 High	1 WordPress Server-Side Request Forgery
 High	1 WordPress Uncontrolled Resource Consumption Vulnerability
 High	1 WordPress Uncontrolled Resource Consumption Vulnerability
 High	1 WordPress Use of Insufficiently Random Values Vulnerability
 High	1 WordPress Weak Password Recovery Mechanism for Forgotten Password Vulnerability
 Medium	1 Directory listings
 Medium	1 Insecure HTTP Usage
 Medium	1 SSL/TLS Not Implemented
 Medium	1 WordPress Cross-Site Request Forgery (CSRF) Vulnerability
 Medium	1 WordPress Cross-Site Request Forgery (CSRF) Vulnerability
 Medium	1 WordPress CVE-2020-25286 Vulnerability
 Medium	1 WordPress Exposure of Sensitive Information to an Unauthorized Actor Vulnerability
 Medium	1 WordPress Exposure of Sensitive Information to an Unauthorized Actor Vulnerability
 Medium	1 WordPress Improper Authentication Vulnerability
 Medium	1 WordPress Improper Input Validation Vulnerability
 Medium	1 WordPress Improper Input Validation Vulnerability
 Medium	1 WordPress Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') Vulnerability

SEVERITY	IMPACT
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability
Medium	1 WordPress Improper Neutralization of Script-Related HTML Tags in a Web Page (Basic XSS) Vulnerability
Medium	1 WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability
Medium	1 WordPress Improper Privilege Management Vulnerability
Medium	1 WordPress Incorrect Authorization Vulnerability
Medium	1 WordPress Other Vulnerability
Medium	1 WordPress Other Vulnerability
Medium	1 WordPress Time-of-check Time-of-use (TOCTOU) Race Condition Vulnerability
Medium	1 WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability
Medium	1 WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability
Medium	1 WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability
Medium	1 WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability

SEVERITY	IMPACT
Medium	1 WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability
Medium	1 WordPress username enumeration
Medium	1 WordPress XML-RPC authentication brute force
Low	1 Broken Link Hijacking
Low	1 Cookies Not Marked as HttpOnly
Low	1 Cookies with missing, inconsistent or contradictory properties
Low	1 Documentation files
Low	1 Missing Content-Type Header
Low	1 Possible sensitive directories
Low	1 WordPress admin accessible without HTTP authentication
Low	1 WordPress Authentication Bypass Using an Alternate Path or Channel Vulnerability
Low	1 WordPress default administrator account
Low	1 WordPress Improper Neutralization of Script-Related HTML Tags in a Web Page (Basic XSS) Vulnerability
Low	1 WordPress REST API User Enumeration

WordPress CVE-2020-28039 Vulnerability

is_protected_meta in wp-includes/meta.php in WordPress before 5.5.2 allows arbitrary file deletion because it does not properly determine whether a meta key is considered protected.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28039](https://nvd.nist.gov/vuln/detail/CVE-2020-28039)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28039>

WordPress Deserialization of Untrusted Data Vulnerability

In WordPress before 4.9.9 and 5.x before 5.0.1, contributors could conduct PHP object injection attacks via crafted metadata in a wp.getMediaItem XMLRPC call. This is caused by mishandling of serialized data at phar:// URLs in the wp_get_attachment_thumb_file function in wp-includes/post.php.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-20148](https://nvd.nist.gov/vuln/detail/CVE-2018-20148)

<https://nvd.nist.gov/vuln/detail/CVE-2018-20148>

WordPress Deserialization of Untrusted Data Vulnerability

PHPMailer 6.1.8 through 6.4.0 allows object injection through Phar Deserialization via addAttachment with a UNC pathname. NOTE: this is similar to CVE-2018-19296, but arose because 6.1.8 fixed a functionality problem in which UNC pathnames were always considered unreadable by PHPMailer, even in safe contexts. As an unintended side effect, this fix eliminated the code that blocked addAttachment exploitation. WordPress Source: <https://wordpress.org/news/2021/05/wordpress-5-7-2-security-release/>

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-36326](https://nvd.nist.gov/vuln/detail/CVE-2020-36326)

<https://nvd.nist.gov/vuln/detail/CVE-2020-36326>

WordPress Deserialization of Untrusted Data Vulnerability

WordPress before 5.5.2 mishandles deserialization requests in wp-includes/Requests/Utility/FilteredIterator.php.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28032](https://nvd.nist.gov/vuln/detail/CVE-2020-28032)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28032>

WordPress Improper Input Validation Vulnerability

wpkses_bad_protocol in wp-includes/kses.php in WordPress before 5.3.1 mishandles the HTML5 colon named entity, allowing attackers to bypass input sanitization, as demonstrated by the javascript: substring.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-20041](https://nvd.nist.gov/vuln/detail/CVE-2019-20041)

<https://nvd.nist.gov/vuln/detail/CVE-2019-20041>

WordPress Improper Input Validation Vulnerability

is_blog_installed in wp-includes/functions.php in WordPress before 5.5.2 improperly determines whether WordPress is already installed, which might allow an attacker to perform a new installation, leading to remote code execution (as well as a denial of service for the old installation).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28037](https://nvd.nist.gov/vuln/detail/CVE-2020-28037)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28037>

WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability

WordPress before 4.8.3 is affected by an issue where \$wpdb->prepare() can create unexpected and unsafe queries leading to potential SQL injection (SQLi) in plugins and themes, as demonstrated by a "double prepare" approach, a different vulnerability than CVE-2017-14723.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-16510](https://nvd.nist.gov/vuln/detail/CVE-2017-16510)

<https://nvd.nist.gov/vuln/detail/CVE-2017-16510>

WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability

Before version 4.8.2, WordPress mishandled % characters and additional placeholder values in \$wpdb->prepare, and thus did not properly address the possibility of plugins and themes enabling SQL injection attacks.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-14723](https://nvd.nist.gov/vuln/detail/CVE-2017-14723)

<https://nvd.nist.gov/vuln/detail/CVE-2017-14723>

WordPress Improper Privilege Management Vulnerability

WordPress before 5.5.2 allows attackers to gain privileges via XML-RPC.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28035](https://nvd.nist.gov/vuln/detail/CVE-2020-28035)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28035>

WordPress Improper Privilege Management Vulnerability

wp-includes/class-wp-xmlrpc-server.php in WordPress before 5.5.2 allows attackers to gain privileges by using XML-RPC to comment on a post.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28036](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28036>

WordPress Other Vulnerability

WordPress before 5.8 lacks support for the Update URI plugin header. This makes it easier for remote attackers to execute arbitrary code via a supply-chain attack against WordPress installations that use any plugin for which the slug satisfies the naming constraints of the WordPress.org Plugin Directory but is not yet present in that directory.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2021-44223](#)

<https://nvd.nist.gov/vuln/detail/CVE-2021-44223>

WordPress Server-Side Request Forgery (SSRF) Vulnerability

WordPress before 5.2.4 has a Server Side Request Forgery (SSRF) vulnerability because Windows paths are mishandled during certain validation of relative URLs.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-17670](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-17670>

WordPress Server-Side Request Forgery (SSRF) Vulnerability

WordPress before 5.2.4 has a Server Side Request Forgery (SSRF) vulnerability because URL validation does not consider the interpretation of a name as a series of hex characters.

Impact

<http://192.168.0.103/>

References

[CVE-2019-17669](https://nvd.nist.gov/vuln/detail/CVE-2019-17669)

<https://nvd.nist.gov/vuln/detail/CVE-2019-17669>

WordPress 4.7.x Arbitrary File Deletion Vulnerability

WordPress is prone to a vulnerability that lets attackers delete arbitrary files because the application fails to properly verify user-supplied input. An attacker can exploit this vulnerability to delete arbitrary files in the context of the webserver process, rendering the website unusable, or even to execute arbitrary code on the webserver. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.10 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.11 or latest

References

<https://blog.ripstech.com/2018/wordpress-file-delete-to-code-execution/>

<https://blog.ripstech.com/2018/wordpress-file-delete-to-code-execution/>

<http://blog.vulnspy.com/2018/06/27/Wordpress-4-9-6-Arbitrary-File-Delection-Vulnerability-Exploit/>

<http://blog.vulnspy.com/2018/06/27/Wordpress-4-9-6-Arbitrary-File-Delection-Vulnerability-Exploit/>

<https://www.exploit-db.com/exploits/44949/>

<https://www.exploit-db.com/exploits/44949/>

<https://packetstormsecurity.com/files/148333/WordPress-4.9.6-Arbitrary-File-Deletion.html>

<https://packetstormsecurity.com/files/148333/WordPress-4.9.6-Arbitrary-File-Deletion.html>

<https://www.wordfence.com/blog/2018/07/details-of-an-additional-file-deletion-vulnerability-patched-in-wordpress-4-9-7/>

<https://www.wordfence.com/blog/2018/07/details-of-an-additional-file-deletion-vulnerability-patched-in-wordpress-4-9-7/>

<http://blog.nsfocus.net/wordpress-delete-any-file/>

<http://blog.nsfocus.net/wordpress-delete-any-file/>

https://codex.wordpress.org/Version_4.7.11

https://codex.wordpress.org/Version_4.7.11

WordPress 4.7.x Cross-Site Request Forgery

WordPress is prone to a cross-site request forgery vulnerability. Exploiting this issue may allow a remote attacker to perform certain administrative actions and gain unauthorized access to the affected application; other attacks are also possible. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.12 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.13 or latest

References

<https://blog.ripstech.com/2019/wordpress-csrf-to-rce/>
https://blog.ripstech.com/2019/wordpress-csrf-to-rce/

<https://cert.360.cn/warning/detail?id=149a32b8f582ee5e0cbd5f1c1b4a61de>
https://cert.360.cn/warning/detail?id=149a32b8f582ee5e0cbd5f1c1b4a61de

<https://wordpress.org/support/wordpress-version/version-4-7-13/>
https://wordpress.org/support/wordpress-version/version-4-7-13/

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, shortcode execution, information disclosure and Denial of Service via cache poisoning vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, to execute shortcodes by calling the built-in 'parse-media-shortcode' AJAX handler, to obtain sensitive information or to deny REST API endpoint access to legitimate users. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.26 are vulnerable.

Impact

<http://192.168.0.103/>
wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.27 or latest

References

<https://wpscan.com/blog/email-leak-oracle-vulnerability-addressed-in-wordpress-6-3-2/>
https://wpscan.com/blog/email-leak-oracle-vulnerability-addressed-in-wordpress-6-3-2/

<https://patchstack.com/articles/wordpress-core-6-3-2-security-update-technical-advisory>
https://patchstack.com/articles/wordpress-core-6-3-2-security-update-technical-advisory

<https://www.wordfence.com/blog/2023/10/wordpress-6-3-2-security-release-what-you-need-to-know/>
https://www.wordfence.com/blog/2023/10/wordpress-6-3-2-security-release-what-you-need-to-know/

<https://wordpress.org/documentation/wordpress-version/version-4-7-27/>
https://wordpress.org/documentation/wordpress-version/version-4-7-27/

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting and security bypass vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, or to bypass certain security restrictions and perform unauthorized actions. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.16 are vulnerable.

Impact

<http://192.168.0.103/>
wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.17 or latest

References

<https://www.wordfence.com/blog/2020/04/unpacking-the-7-vulnerabilities-fixed-in-todays-wordpress-5-4-1-security-update/>
<https://www.wordfence.com/blog/2020/04/unpacking-the-7-vulnerabilities-fixed-in-todays-wordpress-5-4-1-security-update/>
<https://wordpress.org/support/wordpress-version/version-4.7.17/>
<https://wordpress.org/support/wordpress-version/version-4.7.17/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, open redirect and security bypass vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, to redirect users to arbitrary web sites and conduct phishing attacks, or to bypass certain security restrictions and perform unauthorized actions. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.17 are vulnerable.

Impact

<http://192.168.0.103/>
wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.18 or latest

References

<https://pentest.co.uk/labs/research/subtle-stored-xss-wordpress-core/>
<https://pentest.co.uk/labs/research/subtle-stored-xss-wordpress-core/>
<https://www.youtube.com/watch?v=tCh7Y8z8fb4>
<https://www.youtube.com/watch?v=tCh7Y8z8fb4>
<https://wordpress.org/support/wordpress-version/version-4.7.18/>
<https://wordpress.org/support/wordpress-version/version-4.7.18/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, cross-site request forgery, open redirect, information disclosure and SQL injection vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, to perform certain administrative actions and gain unauthorized access to the affected application, to redirect users to arbitrary web sites and conduct phishing attacks, to obtain potentially sensitive information or to compromise the application, access or modify data or exploit vulnerabilities in the underlying database. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.24 are vulnerable.

Impact

<http://192.168.0.103/>
wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.25 or latest

References

<https://www.wordfence.com/blog/2022/10/patch-now-the-wordpress-6-0-3-security-update-contains-important-fixes/>

<https://www.wordfence.com/blog/2022/10/patch-now-the-wordpress-6-0-3-security-update-contains-important-fixes/>
<https://wordpress.org/support/wordpress-version/version-4-7-25/>
<https://wordpress.org/support/wordpress-version/version-4-7-25/>
<https://jvn.jp/en/jp/JVN09409909/index.html>
<https://jvn.jp/en/jp/JVN09409909/index.html>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting and security bypass vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, or to bypass certain security restrictions and perform unauthorized actions. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.15 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.16 or latest

References

<https://github.com/WordPress/wordpress-develop/security/advisories/GHSA-g7rg-hchx-c2gw>
<https://github.com/WordPress/wordpress-develop/security/advisories/GHSA-g7rg-hchx-c2gw>
<https://github.com/WordPress/wordpress-develop/security/advisories/GHSA-pg4x-64rh-3c9v>
<https://github.com/WordPress/wordpress-develop/security/advisories/GHSA-pg4x-64rh-3c9v>
<https://github.com/WordPress/wordpress-develop/security/advisories/GHSA-x3wp-h3qx-9w94>
<https://github.com/WordPress/wordpress-develop/security/advisories/GHSA-x3wp-h3qx-9w94>
<https://wordpress.org/support/wordpress-version/version-4-7-16/>
<https://wordpress.org/support/wordpress-version/version-4-7-16/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, security bypass, information disclosure and PHP object injection vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials and launch other attacks, to perform otherwise restricted actions and subsequently delete files that they weren't authorized to or create posts of unauthorized post types, to obtain sensitive information, or to possibly execute arbitrary PHP code within the context of the affected webserver process. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.11 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.12 or latest

References

<https://cdn2.hubspot.net/hubfs/3853213/us-18-Thomas-It's-A-PHP-Unserialization-Vulnerability-Jim-But-Not-As-We-....pdf?>
<https://cdn2.hubspot.net/hubfs/3853213/us-18-Thomas-It's-A-PHP-Unserialization-Vulnerability-Jim-But-Not-As-We-....pdf?>

<https://blog.ripstech.com/2018/wordpress-post-type-privilege-escalation/>
<https://blog.ripstech.com/2018/wordpress-post-type-privilege-escalation/>

<https://blog.ripstech.com/2019/wordpress-image-remote-code-execution/>
<https://blog.ripstech.com/2019/wordpress-image-remote-code-execution/>

<http://foreversong.cn/archives/1364>
<http://foreversong.cn/archives/1364>

<https://wordpress.org/support/wordpress-version/version-4-7-12/>
<https://wordpress.org/support/wordpress-version/version-4-7-12/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting and open redirect vulnerabilities. An attacker may leverage these issues to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, or to redirect users to arbitrary web sites and conduct phishing attacks. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.13 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.14 or latest

References

<https://www.wordfence.com/blog/2019/09/the-wordpress-5-2-3-security-release-unpacked/>
<https://www.wordfence.com/blog/2019/09/the-wordpress-5-2-3-security-release-unpacked/>

<https://www.exploit-db.com/exploits/49338>
<https://www.exploit-db.com/exploits/49338>

<https://packetstormsecurity.com/files/160745/WordPress-Core-5.2.2-Cross-Site-Scripting.html>
<https://packetstormsecurity.com/files/160745/WordPress-Core-5.2.2-Cross-Site-Scripting.html>

<https://wordpress.org/support/wordpress-version/version-4-7-14/>
<https://wordpress.org/support/wordpress-version/version-4-7-14/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including XML External Entity injection and information disclosure vulnerabilities. Exploiting these issues could allow an attacker to obtain sensitive information which could be used to launch further attacks. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.19 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.20 or latest

References

<https://blog.sonarsource.com/wordpress-xxe-security-vulnerability/>

<https://blog.sonarsource.com/wordpress-xxe-security-vulnerability/>

<https://github.com/motikan2010/CVE-2021-29447>

<https://github.com/motikan2010/CVE-2021-29447>

<https://wordpress.org/support/wordpress-version/version-4-7-20/>

<https://wordpress.org/support/wordpress-version/version-4-7-20/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting and SQL injection vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, or to compromise the application, access or modify data or exploit vulnerabilities in the underlying database. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.23 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.24 or latest

References

<https://www.wordfence.com/blog/2022/08/wordpress-core-6-0-2-security-maintenance-release-what-you-need-to-know/>

<https://www.wordfence.com/blog/2022/08/wordpress-core-6-0-2-security-maintenance-release-what-you-need-to-know/>

<https://wordpress.org/news/2022/08/wordpress-6-0-2-security-and-maintenance-release/>

<https://wordpress.org/news/2022/08/wordpress-6-0-2-security-and-maintenance-release/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, security bypass, or server-side request forgery vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, to bypass certain security restrictions and perform unauthorized actions, or to make the vulnerable server perform port scanning of hosts in internal or external networks. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.14 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.15 or latest

References

<https://blog.wpscan.org/wordpress/security/release/2019/10/15/wordpress-524-security-release-breakdown.html>

<https://blog.wpscan.org/wordpress/security/release/2019/10/15/wordpress-524-security-release-breakdown.html>

<https://Oday.work/proof-of-concept-for-wordpress-5-2-3-viewing-unauthenticated-posts/>

<https://Oday.work/proof-of-concept-for-wordpress-5-2-3-viewing-unauthenticated-posts/>

<https://blog.ripstech.com/2020/wordpress-hardening-bypass/>

<https://blog.ripstech.com/2020/wordpress-hardening-bypass/>

<https://wordpress.org/support/wordpress-version/version-4-7-15/>

<https://wordpress.org/support/wordpress-version/version-4-7-15/>

<https://wordpress.org/news/2019/11/wordpress-5-2-4-update/>

<https://wordpress.org/news/2019/11/wordpress-5-2-4-update/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, SQL injection and PHP object injection vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, to compromise the application, access or modify data, or exploit latent vulnerabilities in the underlying database, or to possibly execute arbitrary PHP code within the context of the affected webserver process.

WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.21 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.22 or latest

References

<https://blog.sonarsource.com/wordpress-stored-xss-vulnerability>

<https://blog.sonarsource.com/wordpress-stored-xss-vulnerability>

<https://www.wordfence.com/blog/2022/01/wordpress-5-8-3-security-release/>

<https://www.wordfence.com/blog/2022/01/wordpress-5-8-3-security-release/>

<https://www.zerodayinitiative.com/advisories/ZDI-22-020/>

<https://www.zerodayinitiative.com/advisories/ZDI-22-020/>

<https://www.exploit-db.com/exploits/50663>

<https://www.exploit-db.com/exploits/50663>

<https://wordpress.org/support/wordpress-version/version-4.7.22/>

<https://wordpress.org/support/wordpress-version/version-4.7.22/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, privilege escalation, security bypass, Denial of Service and PHP object injection vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials and launch other attacks, to bypass the expected capabilities check, to perform otherwise restricted actions and subsequently delete arbitrary files, to deny service to legitimate users, or to possibly execute arbitrary PHP code within the context of the affected webserver process. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.18 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.19 or latest

References

<https://blog.sucuri.net/2020/10/reflected-xss-in-wordpress-v5-5-1-and-lower.html>

<https://blog.sucuri.net/2020/10/reflected-xss-in-wordpress-v5-5-1-and-lower.html>

<https://blog.wpscan.com/2020/10/30/wordpress-5.5.2-security-release.html>

<https://blog.wpscan.com/2020/10/30/wordpress-5.5.2-security-release.html>

<https://threatpost.com/wordpress-patches-rce-bug/160812/>

<https://threatpost.com/wordpress-patches-rce-bug/160812/>

<https://wordpress.org/support/wordpress-version/version-4-7-19/>

<https://wordpress.org/support/wordpress-version/version-4-7-19/>

WordPress 4.7.x Multiple Vulnerabilities

WordPress is prone to multiple vulnerabilities, including cross-site scripting, cross-site request forgery and directory traversal vulnerabilities. Exploiting these issues could allow an attacker to execute arbitrary script code in the browser of an unsuspecting user in the context of the affected site, allowing the attacker to steal cookie-based authentication credentials, to perform certain administrative actions and gain unauthorized access to the affected application, or to obtain sensitive information that may help in launching further attacks. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.25 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.26 or latest

References

<https://patchstack.com/articles/wordpress-core-6-2-1-security-update-technical-advisory/>

<https://patchstack.com/articles/wordpress-core-6-2-1-security-update-technical-advisory/>

<https://www.wordfence.com/blog/2023/05/wordpress-core-6-2-1-security-maintenance-release-what-you-need-to-know/>

<https://www.wordfence.com/blog/2023/05/wordpress-core-6-2-1-security-maintenance-release-what-you-need-to-know/>

<https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/>

<https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-security-release/>

WordPress 4.7.x PHP Object Injection

WordPress is prone to a vulnerability that lets remote attackers inject and execute arbitrary code because the application fails to sanitize user-supplied input before being passed to the unserialize() PHP function. Attackers can possibly exploit this issue to execute arbitrary PHP code within the context of the affected webserver process. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.20 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.21 or latest

References

<https://github.com/JamesGeee/CVE-2020-36326>

<https://github.com/JamesGeee/CVE-2020-36326>

<https://wordpress.org/support/wordpress-version/version-4-7-21/>

<https://wordpress.org/support/wordpress-version/version-4-7-21/>

WordPress 4.7.x Prototype Pollution

WordPress is prone to a prototype pollution vulnerability. Exploiting this issue could allow an attacker to inject key/value properties into JavaScript objects, potentially allowing for execution of arbitrary JavaScript in a user's session if they can trick that user into clicking a link. WordPress versions 4.7.x ranging from 4.7 and up to (and including) 4.7.22 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Update to WordPress version 4.7.23 or latest

References

<https://github.com/BlackFan/client-side-prototype-pollution/blob/master/pp/jquery-query-object.md>

<https://github.com/BlackFan/client-side-prototype-pollution/blob/master/pp/jquery-query-object.md>

<https://wordpress.org/support/wordpress-version/version-4.7.23/>

<https://wordpress.org/support/wordpress-version/version-4.7.23/>

WordPress Access of Resource Using Incompatible Type ('Type Confusion') Vulnerability

WordPress before 5.2.4 does not properly consider type confusion during validation of the referer in the admin pages, possibly leading to CSRF.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-17675](https://nvd.nist.gov/vuln/detail/CVE-2019-17675)

<https://nvd.nist.gov/vuln/detail/CVE-2019-17675>

WordPress Cross-Site Request Forgery (CSRF) Vulnerability

WordPress before 5.1.1 does not properly filter comment content, leading to Remote Code Execution by unauthenticated users in a default configuration. This occurs because CSRF protection is mishandled, and because Search Engine Optimization of A elements is performed incorrectly, leading to XSS. The XSS results in administrative access, which allows arbitrary changes to .php files. This is related to wp-admin/includes/ajax-actions.php and wp-includes/comment.php.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-9787](https://nvd.nist.gov/vuln/detail/CVE-2019-9787)

<https://nvd.nist.gov/vuln/detail/CVE-2019-9787>

WordPress CVE-2019-17673 Vulnerability

WordPress before 5.2.4 is vulnerable to poisoning of the cache of JSON GET requests because certain requests lack a Vary: Origin header.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-17673](https://nvd.nist.gov/vuln/detail/CVE-2019-17673)

<https://nvd.nist.gov/vuln/detail/CVE-2019-17673>

WordPress CVE-2020-28033 Vulnerability

WordPress before 5.5.2 mishandles embeds from disabled sites on a multisite network, as demonstrated by allowing a spam embed.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28033](https://nvd.nist.gov/vuln/detail/CVE-2020-28033)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28033>

WordPress Deserialization of Untrusted Data Vulnerability

PHPMailer before 5.2.27 and 6.x before 6.0.6 is vulnerable to an object injection attack.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-19296](https://nvd.nist.gov/vuln/detail/CVE-2018-19296)

<https://nvd.nist.gov/vuln/detail/CVE-2018-19296>

WordPress Deserialization of Untrusted Data Vulnerability

WordPress is a free and open-source content management system written in PHP and paired with a MariaDB database. On a multisite, users with Super Admin role can bypass explicit/additional hardening under certain conditions through object injection. This has been patched in WordPress version 5.8.3. Older affected versions are also fixed via security release, that go back till 3.7.37. We strongly recommend that you keep auto-updates enabled. There are no known workarounds for this issue.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2022-21663](https://nvd.nist.gov/vuln/detail/CVE-2022-21663)

<https://nvd.nist.gov/vuln/detail/CVE-2022-21663>

WordPress Directory Traversal

WordPress is prone to a directory traversal vulnerability because it fails to sufficiently verify user-supplied input data. Exploiting this issue may allow an attacker to access sensitive information that could aid in further attacks. WordPress versions ranging from 3.7 and up to (and including) 5.0.3 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Edit the source code to ensure that input is properly verified, or apply a fix when available

References

<https://blog.ripstech.com/2019/wordpress-image-remote-code-execution/>

<https://blog.ripstech.com/2019/wordpress-image-remote-code-execution/>

<http://foreversong.cn/archives/1364>

<http://foreversong.cn/archives/1364>

WordPress Exposure of Sensitive Information to an Unauthorized Actor Vulnerability

In WordPress before 4.9.9 and 5.x before 5.0.1, the user-activation page could be read by a search engine's web crawler if an unusual configuration were chosen. The search engine could then index and display a user's e-mail address and (rarely) the password that was generated by default.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-20151](https://nvd.nist.gov/vuln/detail/CVE-2018-20151)

<https://nvd.nist.gov/vuln/detail/CVE-2018-20151>

WordPress Improper Control of Generation of Code ('Code Injection') Vulnerability

WordPress before 4.9.9 and 5.x before 5.0.1 allows remote code execution because an `_wp_attached_file` Post Meta entry can be changed to an arbitrary string, such as one ending with a `.jpg?file.php` substring. An attacker with author privileges can execute arbitrary code by uploading a crafted image containing PHP code in the Exif metadata. Exploitation can leverage CVE-2019-8943.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-8942](https://nvd.nist.gov/vuln/detail/CVE-2019-8942)

<https://nvd.nist.gov/vuln/detail/CVE-2019-8942>

WordPress Improper Input Validation Vulnerability

The Dynamic OOO widget for the Elementor Pro plugin through 3.0.5 for WordPress allows remote authenticated users to execute arbitrary code because only the Editor role is needed to upload executable PHP code via the PHP Raw snippet. NOTE: this issue can be mitigated by removing the Dynamic OOO widget or by restricting availability of the Editor role.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-26596](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-26596>

WordPress Improper Input Validation Vulnerability

WordPress version <4.9 contains a CWE-20 Input Validation vulnerability in thumbnail processing that can result in remote code execution. This attack appears to be exploitable via thumbnail upload by an authenticated user and may require additional plugins in order to be exploited however this has not been confirmed at this time. This issue appears to have been partially, but not completely fixed in WordPress 4.9

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-1000600](#)

<https://nvd.nist.gov/vuln/detail/CVE-2017-1000600>

WordPress Improper Input Validation Vulnerability

WordPress version 4.9.8 and earlier contains a CWE-20 Input Validation vulnerability in thumbnail processing that can result in remote code execution due to an incomplete fix for CVE-2017-1000600. This attack appears to be exploitable via thumbnail upload by an authenticated user and may require additional plugins in order to be exploited however this has not been confirmed at this time.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-1000773](#)

<https://nvd.nist.gov/vuln/detail/CVE-2018-1000773>

WordPress Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') Vulnerability

WordPress through 4.9.6 allows Author users to execute arbitrary code by leveraging directory traversal in the wp-admin/post.php thumb parameter, which is passed to the PHP unlink function and can delete the wp-config.php file. This is related to missing filename validation in the wp-includes/post.php wp_delete_attachment function. The attacker must have capabilities for files and posts that are normally available only to the Author, Editor, and Administrator roles. The attack methodology is to delete wp-config.php and then launch a new installation process to increase the attacker's privileges.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-12895](https://nvd.nist.gov/vuln/detail/CVE-2018-12895)

<https://nvd.nist.gov/vuln/detail/CVE-2018-12895>

WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability

SQL injection vulnerability in stnl_iframe.php in the ShiftThis Newsletter (st_newsletter) plugin for WordPress allows remote attackers to execute arbitrary SQL commands via the newsletter parameter, a different vector than CVE-2008-0683.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2008-4625](https://nvd.nist.gov/vuln/detail/CVE-2008-4625)

<https://nvd.nist.gov/vuln/detail/CVE-2008-4625>

WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability

SQL injection vulnerability in fim_rss.php in the fGallery 2.4.1 plugin for WordPress allows remote attackers to execute arbitrary SQL commands via the album parameter.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2008-0491](https://nvd.nist.gov/vuln/detail/CVE-2008-0491)

<https://nvd.nist.gov/vuln/detail/CVE-2008-0491>

WordPress Inadequate Encryption Strength Vulnerability

WordPress through 4.8.2 uses a weak MD5-based password hashing algorithm, which makes it easier for attackers to determine cleartext values by leveraging access to the hash values. NOTE: the approach to changing this may not be fully compatible with certain use cases, such as migration of a WordPress site from a web host that uses a recent PHP version to a different web host that uses PHP 5.2. These use cases are plausible (but very unlikely) based on statistics showing widespread deployment of WordPress with obsolete PHP versions.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2012-6707](https://nvd.nist.gov/vuln/detail/CVE-2012-6707)

<https://nvd.nist.gov/vuln/detail/CVE-2012-6707>

WordPress Missing Authentication for Critical Function Vulnerability

In affected versions of WordPress, some private posts, which were previously public, can result in unauthenticated disclosure under a specific set of conditions. This has been patched in version 5.4.1, along with all the previously affected versions via a minor release (5.3.3, 5.2.6, 5.1.5, 5.0.9, 4.9.14, 4.8.13, 4.7.17, 4.6.18, 4.5.21, 4.4.22, 4.3.23, 4.2.27, 4.1.30, 4.0.30, 3.9.31, 3.8.33, 3.7.33).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-11028](https://nvd.nist.gov/vuln/detail/CVE-2020-11028)

<https://nvd.nist.gov/vuln/detail/CVE-2020-11028>

WordPress Server-Side Request Forgery

WordPress is prone to a server-side request forgery vulnerability. An attacker may leverage this issue to make the vulnerable server perform port scanning of hosts in internal or external networks; other attacks are also possible. WordPress versions ranging from 3.7 and up to (and including) 6.1.1 are vulnerable.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

Recommendation

Block/Turn off access to XMLRPC/pingbacks as per researchers recommendation

References

<https://blog.sonarsource.com/wordpress-core-unauthenticated-blind-ssrf/>

<https://blog.sonarsource.com/wordpress-core-unauthenticated-blind-ssrf/>

<https://sploit.us/exploit?id=WPEX-ID:C8814E6E-78B3-4F63-A1D3-6906A84C1F11>

<https://sploit.us/exploit?id=WPEX-ID:C8814E6E-78B3-4F63-A1D3-6906A84C1F11>

WordPress Uncontrolled Resource Consumption Vulnerability

In WordPress through 4.9.2, unauthenticated attackers can cause a denial of service (resource consumption) by using the large list of registered .js files (from wp-includes/script-loader.php) to construct a series of requests to load every file many times.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-6389](https://nvd.nist.gov/vuln/detail/CVE-2018-6389)

<https://nvd.nist.gov/vuln/detail/CVE-2018-6389>

WordPress Uncontrolled Resource Consumption Vulnerability

WordPress through 6.1.1 depends on unpredictable client visits to cause wp-cron.php execution and the resulting security updates, and the source code describes "the scenario where a site may not receive enough visits to execute scheduled tasks in a timely manner," but neither the installation guide nor the security guide mentions this default behavior, or alerts the user about security risks on installations with very few visits.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2023-22622](https://nvd.nist.gov/vuln/detail/CVE-2023-22622)

<https://nvd.nist.gov/vuln/detail/CVE-2023-22622>

WordPress Use of Insufficiently Random Values Vulnerability

wp-admin/user-new.php in WordPress before 4.9.1 sets the newbloguser key to a string that can be directly derived from the user ID, which allows remote attackers to bypass intended access restrictions by entering this string.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-17091](https://nvd.nist.gov/vuln/detail/CVE-2017-17091)

<https://nvd.nist.gov/vuln/detail/CVE-2017-17091>

WordPress Weak Password Recovery Mechanism for Forgotten Password Vulnerability

In affected versions of WordPress, a password reset link emailed to a user does not expire upon changing the user password. Access would be needed to the email account of the user by a malicious party for successful execution. This has been patched in version 5.4.1, along with all the previously affected versions via a minor release (5.3.3, 5.2.6, 5.1.5, 5.0.9, 4.9.14, 4.8.13, 4.7.17, 4.6.18, 4.5.21, 4.4.22, 4.3.23, 4.2.27, 4.1.30, 4.0.30, 3.9.31, 3.8.33, 3.7.33).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-11027](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-11027>

Directory listings

Directory listing is a web server function that displays the directory contents when there is no index file in a specific website directory. It is dangerous to leave this function turned on for the web server because it leads to information disclosure.

Impact

A user can view a list of all files from the affected directories possibly exposing sensitive information.

<http://192.168.0.103/>

Verified

Folders with directory listing enabled:

- <http://192.168.0.103/wp-admin/css/>
- <http://192.168.0.103/wp-includes/>
- <http://192.168.0.103/wp-admin/images/>
- <http://192.168.0.103/wp-includes/ID3/>
- <http://192.168.0.103/wp-content/languages/>
- <http://192.168.0.103/wp-includes/IXR/>
- <http://192.168.0.103/wp-includes/Requests/>
- <http://192.168.0.103/wp-admin/includes/>
- <http://192.168.0.103/wp-includes/SimplePie/>
- <http://192.168.0.103/wp-admin/js/>
- <http://192.168.0.103/wp-includes/Text/>
- <http://192.168.0.103/wp-content/languages/plugins/>
- <http://192.168.0.103/wp-content/languages/themes/>
- <http://192.168.0.103/wp-includes/certificates/>
- <http://192.168.0.103/wp-includes/customize/>
- <http://192.168.0.103/wp-includes/fonts/>
- <http://192.168.0.103/wp-includes/pomo/>
- http://192.168.0.103/wp-includes/random_compat/
- <http://192.168.0.103/wp-includes/rest-api/>
- <http://192.168.0.103/wp-includes/theme-compat/>
- <http://192.168.0.103/wp-includes/widgets/>

Request

GET /wp-admin/css/ HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive

Recommendation

You should make sure no sensitive information is disclosed or you may want to restrict directory listings from the web server configuration.

References

[CWE-548: Exposure of Information Through Directory Listing](https://cwe.mitre.org/data/definitions/548.html)
<https://cwe.mitre.org/data/definitions/548.html>

Insecure HTTP Usage

It was detected that your web application uses HTTP protocol, but doesn't automatically redirect users to HTTPS.

Impact

In some circumstances, it could be used for a man-in-the-middle (MitM) attack

<http://192.168.0.103/>

Request

GET / HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive

Recommendation

It's recommended to implement best practices of HTTP Redirection into your web application. Consult web references for more information

References

[HTTP Redirections](https://infosec.mozilla.org/guidelines/web_security#http-redirections)
https://infosec.mozilla.org/guidelines/web_security#http-redirections

SSL/TLS Not Implemented

This scan target was connected to over an unencrypted connection. A potential attacker can intercept and modify data sent and received from this site.

Impact

Possible information disclosure.

<http://192.168.0.103/>

Verified

Request

GET / HTTP/1.1
Referer: http://192.168.0.103/
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive

Recommendation

The site should send and receive data over a secure (HTTPS) connection.

WordPress Cross-Site Request Forgery (CSRF) Vulnerability

WordPress before 5.5.2 allows CSRF attacks that change a theme's background image.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28040](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28040>

WordPress Cross-Site Request Forgery (CSRF) Vulnerability

Multiple cross-site request forgery (CSRF) vulnerabilities in wp-contact-form/options-contactform.php in the WP-ContactForm 1.5 alpha and earlier plugin for WordPress allow remote attackers to perform actions as administrators via the (1) wpcf_question, (2) wpcf_success_msg, or (3) wpcf_error_msg parameter to wp-admin/admin.php.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2008-0198](#)

<https://nvd.nist.gov/vuln/detail/CVE-2008-0198>

WordPress CVE-2020-25286 Vulnerability

In wp-includes/comment-template.php in WordPress before 5.4.2, comments from a post or page could sometimes be seen in the latest comments even if the post or page was not public.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-25286](https://nvd.nist.gov/vuln/detail/CVE-2020-25286)

<https://nvd.nist.gov/vuln/detail/CVE-2020-25286>

WordPress Exposure of Sensitive Information to an Unauthorized Actor Vulnerability

Wordpress is an open source CMS. One of the blocks in the WordPress editor can be exploited in a way that exposes password-protected posts and pages. This requires at least contributor privileges. This has been patched in WordPress 5.7.1, along with the older affected versions via minor releases. It's strongly recommended that you keep auto-updates enabled to receive the fix.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2021-29450](https://nvd.nist.gov/vuln/detail/CVE-2021-29450)

<https://nvd.nist.gov/vuln/detail/CVE-2021-29450>

WordPress Exposure of Sensitive Information to an Unauthorized Actor Vulnerability

In WordPress before 5.2.4, unauthenticated viewing of certain content is possible because the static query property is mishandled.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-17671](https://nvd.nist.gov/vuln/detail/CVE-2019-17671)

<https://nvd.nist.gov/vuln/detail/CVE-2019-17671>

WordPress Improper Authentication Vulnerability

Improper authentication vulnerability in WordPress versions prior to 6.0.3 allows a remote unauthenticated attacker to obtain the email address of the user who posted a blog using the WordPress Post by Email Feature. The developer also provides new patched releases for all versions since 3.7.

Impact

<http://192.168.0.103/>

References

[CVE-2022-43504](#)

<https://nvd.nist.gov/vuln/detail/CVE-2022-43504>

WordPress Improper Input Validation Vulnerability

WordPress through 4.8.2, when domain-based flashmediaelement.swf sandboxing is not used, allows remote attackers to conduct cross-domain Flash injection (XSF) attacks by leveraging code contained within the wp-includes/js/mediaelement/flashmediaelement.swf file.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2016-9263](#)

<https://nvd.nist.gov/vuln/detail/CVE-2016-9263>

WordPress Improper Input Validation Vulnerability

In WordPress before 4.9.9 and 5.x before 5.0.1, authors could bypass intended restrictions on post types via crafted input.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-20152](#)

<https://nvd.nist.gov/vuln/detail/CVE-2018-20152>

WordPress Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') Vulnerability

WordPress Core is vulnerable to Directory Traversal in versions up to, and including, 6.2, via the 'wp_lang' parameter. This allows unauthenticated attackers to access and load arbitrary translation files. In cases where an attacker is able to upload a crafted translation file onto the site, such as via an upload form, this could be also used to perform a Cross-Site Scripting attack.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2023-2745](#)

<https://nvd.nist.gov/vuln/detail/CVE-2023-2745>

WordPress Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') Vulnerability

WordPress through 5.0.3 allows Path Traversal in `wp_crop_image()`. An attacker (who has privileges to crop an image) can write the output image to an arbitrary directory via a filename containing two image extensions and `../` sequences, such as a filename ending with the `.jpg?/../../file.jpg` substring.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-8943](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-8943>

WordPress Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') Vulnerability

Directory traversal vulnerability in `wp-admin/admin.php` in the DMSGuestbook 1.8.0 and 1.7.0 plugin for WordPress allows remote authenticated users to read arbitrary files via a `..` (dot dot) in the (1) folder and (2) file parameters.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2008-0615](#)

<https://nvd.nist.gov/vuln/detail/CVE-2008-0615>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.3 allows reflected XSS in the dashboard.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-16221](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-16221>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Cross-site scripting vulnerability in WordPress versions prior to 6.0.3 allows a remote unauthenticated attacker to inject an arbitrary script. The developer also provides new patched releases for all versions since 3.7.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2022-43500](#)

<https://nvd.nist.gov/vuln/detail/CVE-2022-43500>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Before version 4.8.2, WordPress was vulnerable to a cross-site scripting attack via shortcodes in the TinyMCE visual editor.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-14726](#)

<https://nvd.nist.gov/vuln/detail/CVE-2017-14726>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In affected versions of WordPress, a cross-site scripting (XSS) vulnerability in the navigation section of Customizer allows JavaScript code to be executed. Exploitation requires an authenticated user. This has been patched in version 5.4.1, along with all the previously affected versions via a minor release (5.3.3, 5.2.6, 5.1.5, 5.0.9, 4.9.14, 4.8.13, 4.7.17, 4.6.18, 4.5.21, 4.4.22, 4.3.23, 4.2.27, 4.1.30, 4.0.30, 3.9.31, 3.8.33, 3.7.33).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-11025](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-11025>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.4 is vulnerable to a stored XSS attack to inject JavaScript into STYLE elements.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-17672](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-17672>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In WordPress before 5.3.1, authenticated users with lower privileges (like contributors) can inject JavaScript code in the block editor, which is executed within the dashboard. It can lead to an admin opening the affected post in the editor leading to XSS.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-16781](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-16781>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Before version 4.8.2, WordPress was vulnerable to cross-site scripting in oEmbed discovery.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-14724](#)

<https://nvd.nist.gov/vuln/detail/CVE-2017-14724>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.5.2 allows stored XSS via post slugs.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-28038](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-28038>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In affected versions of WordPress, a special payload can be crafted that can lead to scripts getting executed within the search block of the block editor. This requires an authenticated user with the ability to add content. This has been patched in version 5.4.1, along with all the previously affected versions via a minor release (5.3.3, 5.2.6, 5.1.5, 5.0.9, 4.9.14, 4.8.13, 4.7.17, 4.6.18, 4.5.21, 4.4.22, 4.3.23, 4.2.27, 4.1.30, 4.0.30, 3.9.31, 3.8.33, 3.7.33).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-11030](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-11030>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In wp-includes/formatting.php in WordPress 3.7 to 5.3.0, the function wp_targeted_link_rel() can be used in a particular way to result in a stored cross-site scripting (XSS) vulnerability. This has been patched in WordPress 5.3.1, along with all the previous WordPress versions from 3.7 to 5.3 via a minor release.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-20042](https://nvd.nist.gov/vuln/detail/CVE-2019-20042)

<https://nvd.nist.gov/vuln/detail/CVE-2019-20042>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.3 allows XSS in post previews by authenticated users.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-16223](https://nvd.nist.gov/vuln/detail/CVE-2019-16223)

<https://nvd.nist.gov/vuln/detail/CVE-2019-16223>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In WordPress before 4.9.9 and 5.x before 5.0.1, when the Apache HTTP Server is used, authors could upload crafted files that bypass intended MIME type restrictions, leading to XSS, as demonstrated by a .jpg file without JPEG data.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-20149](https://nvd.nist.gov/vuln/detail/CVE-2018-20149)

<https://nvd.nist.gov/vuln/detail/CVE-2018-20149>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In WordPress before 4.9.9 and 5.x before 5.0.1, crafted URLs could trigger XSS for certain use cases involving plugins.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-20150](https://nvd.nist.gov/vuln/detail/CVE-2018-20150)

<https://nvd.nist.gov/vuln/detail/CVE-2018-20150>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Cross-site scripting (XSS) vulnerability in Peter's Random Anti-Spam Image 0.2.4 and earlier plugin for WordPress allows remote attackers to inject arbitrary web script or HTML via the comment field in the comment form.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2007-6677](https://nvd.nist.gov/vuln/detail/CVE-2007-6677)

<https://nvd.nist.gov/vuln/detail/CVE-2007-6677>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Before version 4.8.2, WordPress allowed Cross-Site scripting in the plugin editor via a crafted plugin name.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-14721](https://nvd.nist.gov/vuln/detail/CVE-2017-14721)

<https://nvd.nist.gov/vuln/detail/CVE-2017-14721>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Cross-site scripting vulnerability in WordPress versions prior to 6.0.3 allows a remote unauthenticated attacker to inject an arbitrary script. The developer also provides new patched releases for all versions since 3.7.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2022-43497](https://nvd.nist.gov/vuln/detail/CVE-2022-43497)

<https://nvd.nist.gov/vuln/detail/CVE-2022-43497>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In affected versions of WordPress, users with low privileges (like contributors and authors) can use the embed block in a certain way to inject unfiltered HTML in the block editor. When affected posts are viewed by a higher privileged user, this could lead to script execution in the editor/wp-admin. This has been patched in version 5.4.2, along with all the previously affected versions via a minor release (5.3.4, 5.2.7, 5.1.6, 5.0.10, 4.9.15, 4.8.14, 4.7.18, 4.6.19, 4.5.22, 4.4.23, 4.3.24, 4.2.28, 4.1.31, 4.0.31, 3.9.32, 3.8.34, 3.7.34).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-4046](https://nvd.nist.gov/vuln/detail/CVE-2020-4046)

<https://nvd.nist.gov/vuln/detail/CVE-2020-4046>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Multiple cross-site scripting (XSS) vulnerabilities in the DMSGuestbook 1.7.0 plugin for WordPress allow remote attackers to inject arbitrary web script or HTML via the (1) file parameter to wp-admin/admin.php, or the (2) messagefield parameter in the guestbook page, and the (3) title parameter in the messagearea.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

wp-includes/feed.php in WordPress before 4.9.1 does not properly restrict enclosures in RSS and Atom fields, which might allow attackers to conduct XSS attacks via a crafted URL.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.3 allows XSS in media uploads because wp_ajax_upload_attachment is mishandled.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.5.2 allows XSS associated with global variables.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.3 has an issue with URL sanitization in wpkses_bad_protocol_once in wp-includes/kses.php that can lead to cross-site scripting (XSS) attacks.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-16222](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-16222>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 4.9.2 has XSS in the Flash fallback files in MediaElement (under wp-includes/js/mediaelement).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-5776](#)

<https://nvd.nist.gov/vuln/detail/CVE-2018-5776>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Before version 4.8.2, WordPress allowed a Cross-Site scripting attack in the template list view via a crafted template name.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-14720](#)

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In affected versions of WordPress, files with a specially crafted name when uploaded to the Media section can lead to script execution upon accessing the file. This requires an authenticated user with privileges to upload files. This has been patched in version 5.4.1, along with all the previously affected versions via a minor release (5.3.3, 5.2.6, 5.1.5, 5.0.9, 4.9.14, 4.8.13, 4.7.17, 4.6.18, 4.5.21, 4.4.22, 4.3.23, 4.2.27, 4.1.30, 4.0.30, 3.9.31, 3.8.33, 3.7.33).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-11026](https://nvd.nist.gov/vuln/detail/CVE-2020-11026)

<https://nvd.nist.gov/vuln/detail/CVE-2020-11026>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Multiple cross-site scripting (XSS) vulnerabilities in the DMSGuestbook 1.8.0 and 1.7.0 plugin for WordPress allow remote attackers to inject arbitrary web script or HTML via the (1) gbname, (2) gbemail, (3) gburl, and (4) gbmsg parameters to unspecified programs. NOTE: the provenance of this information is unknown; the details are obtained solely from third party information.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2008-0618](https://nvd.nist.gov/vuln/detail/CVE-2008-0618)

<https://nvd.nist.gov/vuln/detail/CVE-2008-0618>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.3 allows XSS in stored comments.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-16218](#)
<https://nvd.nist.gov/vuln/detail/CVE-2019-16218>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress users with lower privileges (like contributors) can inject JavaScript code in the block editor using a specific payload, which is executed within the dashboard. This can lead to XSS if an admin opens the post in the editor. Execution of this attack does require an authenticated user. This has been patched in WordPress 5.3.1, along with all the previous WordPress versions from 3.7 to 5.3 via a minor release. Automatic updates are enabled by default for minor releases and we strongly recommend that you keep them enabled.

Impact

<http://192.168.0.103/>
wordpress v4.7.10-4.7.10

References

[CVE-2019-16780](#)
<https://nvd.nist.gov/vuln/detail/CVE-2019-16780>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Before version 4.8.2, WordPress was susceptible to a Cross-Site Scripting attack in the link modal via a javascript: or data: URL.

Impact

<http://192.168.0.103/>
wordpress v4.7.10-4.7.10

References

[CVE-2017-14718](#)
<https://nvd.nist.gov/vuln/detail/CVE-2017-14718>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

wp-includes/functions.php in WordPress before 4.9.1 does not require the unfiltered_html capability for upload of .js files, which might allow remote attackers to conduct XSS attacks via a crafted file.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-17092](#)

<https://nvd.nist.gov/vuln/detail/CVE-2017-17092>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.4 is vulnerable to stored XSS (cross-site scripting) via the Customizer.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-17674](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-17674>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In WordPress before 4.9.9 and 5.x before 5.0.1, contributors could modify new comments made by users with greater privileges, possibly causing XSS.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-20153](#)

<https://nvd.nist.gov/vuln/detail/CVE-2018-20153>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

In affected versions of WordPress, a vulnerability in the stats() method of class-wp-object-cache.php can be exploited to execute cross-site scripting (XSS) attacks. This has been patched in version 5.4.1, along with all the previously affected versions via a minor release (5.3.3, 5.2.6, 5.1.5, 5.0.9, 4.9.14, 4.8.13, 4.7.17, 4.6.18, 4.5.21, 4.4.22, 4.3.23, 4.2.27, 4.1.30, 4.0.30, 3.9.31, 3.8.33, 3.7.33).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-11029](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-11029>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

Before WordPress 4.9.5, the version string was not escaped in the `get_the_generator` function, and could lead to XSS in a generator tag.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-10102](#)

<https://nvd.nist.gov/vuln/detail/CVE-2018-10102>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

`wp-includes/general-template.php` in WordPress before 4.9.1 does not properly restrict the `lang` attribute of an HTML element, which might allow attackers to conduct XSS attacks via the language setting of a site.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-17093](#)

<https://nvd.nist.gov/vuln/detail/CVE-2017-17093>

WordPress Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') Vulnerability

WordPress before 5.2.3 allows XSS in shortcode previews.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-16219](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-16219>

WordPress Improper Neutralization of Script-Related HTML Tags in a Web Page (Basic XSS) Vulnerability

In affected versions of WordPress, authenticated users with upload permissions (like authors) are able to inject JavaScript into some media file attachment pages in a certain way. This can lead to script execution in the context of a higher privileged user when the file is viewed by them. This has been patched in version 5.4.2, along with all the previously affected versions via a minor release (5.3.4, 5.2.7, 5.1.6, 5.0.10, 4.9.15, 4.8.14, 4.7.18, 4.6.19, 4.5.22, 4.4.23, 4.3.24, 4.2.28, 4.1.31, 4.0.31, 3.9.32, 3.8.34, 3.7.34).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-4047](#)

<https://nvd.nist.gov/vuln/detail/CVE-2020-4047>

WordPress Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') Vulnerability

SQL injection vulnerability in the administration panel in the DMSGuestbook 1.7.0 plugin for WordPress allows remote authenticated administrators to execute arbitrary SQL commands via unspecified vectors. NOTE: it is not clear whether this issue crosses privilege boundaries.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2008-0616](#)

<https://nvd.nist.gov/vuln/detail/CVE-2008-0616>

WordPress Improper Privilege Management Vulnerability

In `wp-includes/rest-api/endpoints/class-wp-rest-posts-controller.php` in WordPress 3.7 to 5.3.0, authenticated users who do not have the rights to publish a post are able to mark posts as sticky or unsticky via the REST API. For example, the contributor role does not have such rights, but this allowed them to bypass that. This has been patched in WordPress 5.3.1, along with all the previous WordPress versions from 3.7 to 5.3 via a minor release.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2019-20043](https://nvd.nist.gov/vuln/detail/CVE-2019-20043)

<https://nvd.nist.gov/vuln/detail/CVE-2019-20043>

WordPress Incorrect Authorization Vulnerability

In WordPress before 4.9.9 and 5.x before 5.0.1, authors could modify metadata to bypass intended restrictions on deleting files.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-20147](https://nvd.nist.gov/vuln/detail/CVE-2018-20147)

<https://nvd.nist.gov/vuln/detail/CVE-2018-20147>

WordPress Other Vulnerability

Cross-site scripting (XSS) vulnerability in `sidebar.php` in WordPress, when custom 404 pages that call `get_sidebar` are used, allows remote attackers to inject arbitrary web script or HTML via the query string (`PHP_SELF`), a different vulnerability than CVE-2007-1622.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2007-2627](https://nvd.nist.gov/vuln/detail/CVE-2007-2627)

<https://nvd.nist.gov/vuln/detail/CVE-2007-2627>

WordPress Other Vulnerability

Cross-site scripting (XSS) vulnerability in `index.php` in the Blue Memories theme 1.5 for WordPress allows remote attackers to inject arbitrary web script or HTML via the `s` parameter, possibly a related issue to CVE-2007-2757 and CVE-2007-4014. NOTE: the provenance of this information is

unknown; the details are obtained solely from third party information.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2007-4165](https://nvd.nist.gov/vuln/detail/CVE-2007-4165)

<https://nvd.nist.gov/vuln/detail/CVE-2007-4165>

WordPress Time-of-check Time-of-use (TOCTOU) Race Condition Vulnerability

WordPress is affected by an unauthenticated blind SSRF in the pingback feature. Because of a TOCTOU race condition between the validation checks and the HTTP request, attackers can reach internal hosts that are explicitly forbidden.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2022-3590](https://nvd.nist.gov/vuln/detail/CVE-2022-3590)

<https://nvd.nist.gov/vuln/detail/CVE-2022-3590>

WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability

Before WordPress 4.9.5, the redirection URL for the login page was not validated or sanitized if forced to use HTTPS.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-10100](https://nvd.nist.gov/vuln/detail/CVE-2018-10100)

<https://nvd.nist.gov/vuln/detail/CVE-2018-10100>

WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability

In affected versions of WordPress, due to an issue in `wp_validate_redirect()` and URL sanitization, an arbitrary external link can be crafted leading to unintended/open redirect when clicked. This has been patched in version 5.4.2, along with all the previously affected versions via a minor release (5.3.4, 5.2.7, 5.1.6, 5.0.10, 4.9.15, 4.8.14, 4.7.18, 4.6.19, 4.5.22, 4.4.23, 4.3.24, 4.2.28, 4.1.31, 4.0.31, 3.9.32, 3.8.34, 3.7.34).

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2020-4048](https://nvd.nist.gov/vuln/detail/CVE-2020-4048)

<https://nvd.nist.gov/vuln/detail/CVE-2020-4048>

WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability

Before WordPress 4.9.5, the URL validator assumed URLs with the hostname localhost were on the same host as the WordPress server.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2018-10101](https://nvd.nist.gov/vuln/detail/CVE-2018-10101)

<https://nvd.nist.gov/vuln/detail/CVE-2018-10101>

WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability

Before version 4.8.2, WordPress was susceptible to an open redirect attack in `wp-admin/edit-tag-form.php` and `wp-admin/user-edit.php`.

Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

References

[CVE-2017-14725](https://nvd.nist.gov/vuln/detail/CVE-2017-14725)

<https://nvd.nist.gov/vuln/detail/CVE-2017-14725>

WordPress URL Redirection to Untrusted Site ('Open Redirect') Vulnerability

In WordPress before 5.2.3, validation and sanitization of a URL in `wp_validate_redirect` in `wp-includes/pluggable.php` could lead to an open redirect.

Impact

[http://192.168.0.103/](#)

wordpress v4.7.10-4.7.10

References

[CVE-2019-16220](#)

<https://nvd.nist.gov/vuln/detail/CVE-2019-16220>

WordPress username enumeration

If permalinks are enabled, in many WordPress installations it is possible to enumerate all the WordPress usernames iterating through the author archives. Whenever a post is published, the username or alias is shown as the author. For example, the URL `http://site.com/?author=1` will show all the posts from user id 1. Attackers can abuse this functionality to figure out which usernames are available on the site.

Impact

An attacker can enumerate the WordPress usernames and use this information to conduct brute-force attacks against passwords for these known usernames.

[http://192.168.0.103/](#)

List of WordPress users for `{'_nativeUrl': {'password': '', 'username': '', 'search': '', 'protocol': 'http', 'port': '', 'path': '/', 'origin': 'http://192.168.0.103', 'href': 'http://192.168.0.103/', 'hostname': '192.168.0.103', 'host': '192.168.0.103', 'hash': ''}}`:

```
[ 'admin' ]
```

Request

```
POST //wp-login.php HTTP/1.1
Content-type: application/x-www-form-urlencoded
Cookie: wordpress_test_cookie=WP+Cookie+check
Content-Length: 29
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

```
log=192&pwd=z&wp-submit=Login
```

Recommendation

You can use an `.htaccess` rewrite rule to prevent this disclosure but you should also be sure to use nicknames to avoid disclosing usernames.

```
# Stop WordPress username enumeration vulnerability
```

```
RewriteCond %{REQUEST_URI} ^/$
```

```
RewriteCond %{QUERY_STRING} ^/?author=([0-9]*)
```

```
RewriteRule ^(.*)$ http://yoursite.com/somepage/? [L,R=301]
```

WordPress XML-RPC authentication brute force

WordPress provides an XML-RPC interface via the `xmlrpc.php` script. XML-RPC is remote procedure calling using HTTP as the transport and XML as the encoding. An attacker can abuse this interface to brute force authentication credentials using API calls such as `wp.getUsersBlogs`.

Impact

An attacker can brute force the authentication credentials for your WordPress blog.

<http://192.168.0.103/xmlrpc.php>

Pattern found:

```
<value><string>Incorrect username or password.</string></value>
```

Request

```
POST //xmlrpc.php HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 264
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive

<?xml version="1.0" encoding="iso-8859-1"?>
<methodCall>
<methodName>wp.getUsersBlogs</methodName>
<params>
<param><value><string>admin</string></value></param>
<param><value><string>89475895437895437534987</string></value>
</param>
</params>
</methodCall>
```

Recommendation

It is possible to disable the XML-RPC script if you do not want to use it. Consult references for a WordPress plugin that does that. If you don't want to disable XML-RPC you can monitor for XML-RPC authentication failures with a Web Application Firewall like ModSecurity.

References

[WordPress XML-RPC Brute Force Scanning](https://www.trustwave.com/en-us/resources/blogs/spiderlabs-blog/honeypot-alert-wordpress-xml-rpc-brute-force-scanning/)

<https://www.trustwave.com/en-us/resources/blogs/spiderlabs-blog/honeypot-alert-wordpress-xml-rpc-brute-force-scanning/>

[Prevent XMLRPC](https://wordpress.org/plugins/prevent-xmlrpc/)

<https://wordpress.org/plugins/prevent-xmlrpc/>

[WordPress brute force attack via wp.getUsersBlogs](https://isc.sans.edu/diary/+WordPress+brute+force+attack+via+wp.getUsersBlogs/18427)

<https://isc.sans.edu/diary/+WordPress+brute+force+attack+via+wp.getUsersBlogs/18427>

Broken Link Hijacking

This web application contains resources (scripts or frames) that are loaded from a non-resolving (potentially expired) domain. An attacker can potentially register the expired domain and serve malicious HTML or JavaScript code (hijacking the resource).

Impact

An attacker can potentially register the expired domain and serve a malicious file that will be loaded in the context of your web application.

<http://192.168.0.103/index.php>

This page contains a `<script>` tag with the `src` attribute (<http://dc-2/wp-includes/js/jquery/jquery.js?ver=1.12.4>) pointing to the domain **dc-2**. This domain does currently not resolve to an IP address and can potentially be taken over by an attacker.

Request

```
GET /index.php HTTP/1.1
Referer: http://192.168.0.103/
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
```

Recommendation

Replace the scripts/frames that are loaded from non-resolving domains with new links that are pointing to valid domains.

References

[Broken Link Hijacking](#)

<https://edoverflow.com/2017/broken-link-hijacking/>

Cookies Not Marked as HttpOnly

One or more cookies don't have the HttpOnly flag set. When a cookie is set with the HttpOnly flag, it instructs the browser that the cookie can only be accessed by the server and not by client-side scripts. This is an important security protection for session cookies.

Impact

Cookies can be accessed by client-side scripts.

<http://192.168.0.103/>

Verified

Cookies without HttpOnly flag set:

- <http://192.168.0.103/wp-login.php>

Set-Cookie: wordpress_test_cookie=WP+Cookie+check; path=/

Request

GET /wp-login.php HTTP/1.1
Referer: http://192.168.0.103/readme.html
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive

Recommendation

If possible, you should set the HttpOnly flag for these cookies.

Cookies with missing, inconsistent or contradictory properties

At least one of the following cookies properties causes the cookie to be invalid or incompatible with either a different property of the same cookie, of with the environment the cookie is being used in. Although this is not a vulnerability in itself, it will likely lead to unexpected behavior by the application, which in turn may cause secondary security issues.

Impact

Cookies will not be stored, or submitted, by web browsers.

<http://192.168.0.103/>

Verified

List of cookies with missing, inconsistent or contradictory properties:

- <http://192.168.0.103/wp-login.php>

Cookie was set with:

```
Set-Cookie: wordpress_test_cookie=WP+Cookie+check; path=/
```

This cookie has the following issues:

- Cookie without SameSite attribute.

When cookies lack the SameSite attribute, Web browsers may apply different and sometimes unexpected defaults. It is therefore recommended to add a SameSite attribute with an appropriate value of either "Strict", "Lax", or "None".

Request

```
GET /wp-login.php HTTP/1.1
Referer: http://192.168.0.103/readme.html
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

Recommendation

Ensure that the cookies configuration complies with the applicable standards.

References

[MDN | Set-Cookie](https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie)

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie>

[Securing cookies with cookie prefixes](https://www.sjoerdlangkemper.nl/2017/02/09/cookie-prefixes/)

<https://www.sjoerdlangkemper.nl/2017/02/09/cookie-prefixes/>

[Cookies: HTTP State Management Mechanism](https://tools.ietf.org/html/draft-ietf-httpbis-rfc6265bis-05)

<https://tools.ietf.org/html/draft-ietf-httpbis-rfc6265bis-05>

[SameSite Updates - The Chromium Projects](https://www.chromium.org/updates/same-site)

<https://www.chromium.org/updates/same-site>

[draft-west-first-party-cookies-07: Same-site Cookies](https://tools.ietf.org/html/draft-west-first-party-cookies-07)

<https://tools.ietf.org/html/draft-west-first-party-cookies-07>

Documentation files

One or more documentation files (e.g. readme.txt, changelog.txt, ...) were found. The information contained in these files could help an attacker identify the web application you are using and sometimes the version of the application. It's recommended to remove these files from production systems.

Impact

These files may disclose sensitive information. This information can be used to launch further attacks.

<http://192.168.0.103/>

Documentation files:

- <http://192.168.0.103/readme.html>

File contents (first 100 characters):

```
<!DOCTYPE html>
<html>
<head>
```

```
<meta name="viewport" content="width=device-width" />
<meta http-equ ...
```

- <http://192.168.0.103/license.txt>
File contents (first 100 characters):

```
WordPress - Web publishing software

Copyright 2011-2018 by the contributors

This program is fr ...
```

- <http://192.168.0.103/wp-content/plugins/akismet/readme.txt>
File contents (first 100 characters):
- ```
=== Akismet ===
Contributors: matt, ryan, andy, mdawaffe, tellyworth, josephscott, lessbloat, eoigal ...
```

- <http://192.168.0.103/wp-includes/ID3/readme.txt>  
File contents (first 100 characters):
- ```
////////////////////////////////////
/// getID3() by James Heinrich <in ...
```

- <http://192.168.0.103/wp-includes/ID3/license.txt>
File contents (first 100 characters):
- ```
////////////////////////////////////
/// getID3() by James Heinrich <in ...
```

Request

```
GET /readme.html HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

Recommendation

Remove or restrict access to all documentation file accessible from internet.

# Missing Content-Type Header

These page(s) does not set a Content-Type header value. This value informs the browser what kind of data to expect. If this header is missing, the browser may incorrectly handle the data. This could lead to security problems.

Impact

None

<http://192.168.0.103/> Verified

Pages where the content-type header is not specified:

- [http://192.168.0.103/wp-content/languages/admin-en\\_AU.mo](http://192.168.0.103/wp-content/languages/admin-en_AU.mo)
- [http://192.168.0.103/wp-content/languages/admin-en\\_AU.po](http://192.168.0.103/wp-content/languages/admin-en_AU.po)
- [http://192.168.0.103/wp-content/languages/themes/twentyfifteen-en\\_AU.mo](http://192.168.0.103/wp-content/languages/themes/twentyfifteen-en_AU.mo)
- [http://192.168.0.103/wp-content/languages/themes/twentyfifteen-en\\_AU.po](http://192.168.0.103/wp-content/languages/themes/twentyfifteen-en_AU.po)
- [http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en\\_AU.mo](http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en_AU.mo)
- [http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en\\_AU.po](http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en_AU.po)
- [http://192.168.0.103/wp-content/languages/admin-network-en\\_AU.mo](http://192.168.0.103/wp-content/languages/admin-network-en_AU.mo)
- [http://192.168.0.103/wp-content/languages/admin-network-en\\_AU.po](http://192.168.0.103/wp-content/languages/admin-network-en_AU.po)
- [http://192.168.0.103/wp-content/languages/en\\_AU.mo](http://192.168.0.103/wp-content/languages/en_AU.mo)

- [http://192.168.0.103/wp-content/languages/en\\_AU.po](http://192.168.0.103/wp-content/languages/en_AU.po)
- [http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en\\_AU.mo](http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en_AU.mo)
- [http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en\\_AU.po](http://192.168.0.103/wp-content/languages/themes/twentyseventeen-en_AU.po)

## Request

```
GET /wp-content/languages/admin-en_AU.mo HTTP/1.1
Referer: http://192.168.0.103/wp-content/languages/
Cookie: wordpress_test_cookie=WP+Cookie+check
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## Recommendation

Set a Content-Type header value for these page(s).

# Possible sensitive directories

One or more possibly sensitive directories were found. These resources are not directly linked from the website. This check looks for common sensitive resources like backup directories, database dumps, administration pages, temporary directories. Each one of these directories could help an attacker to learn more about his target.

## Impact

These directories may expose sensitive information that could help a malicious user to prepare more advanced attacks.

## <http://192.168.0.103/>

Possible sensitive directories:

- <http://192.168.0.103/wp-admin/includes>

## Request

```
GET /wp-admin/includes/ HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## Recommendation

Restrict access to these directories or remove them from the website.

## References

[Web Server Security and Database Server Security](https://www.acunetix.com/websitesecurity/webserver-security/)

<https://www.acunetix.com/websitesecurity/webserver-security/>

# WordPress admin accessible without HTTP authentication

It's recommended to restrict access to the WordPress administration dashboard using HTTP authentication. Password protecting your WordPress admin dashboard through a layer of HTTP authentication is an effective measure to thwart attackers attempting to guess user's passwords. Additionally, if attackers manage to steal a user's password, they will need to get past HTTP authentication in order to gain access to WordPress login form.

## Impact

No impact is associated with this vulnerability.

<http://192.168.0.103/wp-admin/>

## Request

```
GET /wp-admin/ HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## Recommendation

Add server-side password protection (such as BasicAuth) to the /wp-admin/ directory. Consult web references for more information.

## References

[Securing wp-admin](https://codex.wordpress.org/Hardening_WordPress)

[https://codex.wordpress.org/Hardening\\_WordPress](https://codex.wordpress.org/Hardening_WordPress)

[WordPress Security Tips Part 5 | Restrict Access to wp-admin Directory](https://www.acunetix.com/blog/articles/wordpress-security-wpadmin-directory/)

<https://www.acunetix.com/blog/articles/wordpress-security-wpadmin-directory/>

# WordPress Authentication Bypass Using an Alternate Path or Channel Vulnerability

In affected versions of WordPress, misuse of the `set-screen-option` filter's return value allows arbitrary user meta fields to be saved. It does require an admin to install a plugin that would misuse the filter. Once installed, it can be leveraged by low privileged users. This has been patched in version 5.4.2, along with all the previously affected versions via a minor release (5.3.4, 5.2.7, 5.1.6, 5.0.10, 4.9.15, 4.8.14, 4.7.18, 4.6.19, 4.5.22, 4.4.23, 4.3.24, 4.2.28, 4.1.31, 4.0.31, 3.9.32, 3.8.34, 3.7.34).

## Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

## References

[CVE-2020-4050](https://nvd.nist.gov/vuln/detail/CVE-2020-4050)

<https://nvd.nist.gov/vuln/detail/CVE-2020-4050>

# WordPress default administrator account

By default WordPress creates an administrator user account named **admin**. Using the default Admin WordPress Account, hackers can easily launch a brute force attack against it. In order to help deter this type of attack, you should change your default WordPress administrator username to something more difficult to guess.

## Impact

No impact is associated with this vulnerability.

<http://192.168.0.103/wp-login.php>

## Request

```
POST //wp-login.php HTTP/1.1
Content-type: application/x-www-form-urlencoded
Content-Length: 41
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

```
log=admin&pwd=testingtest&wp-submit=Login
```

## Recommendation

Change the default WordPress administrator username to something more difficult to guess. Consult web references for more information.

## References

[OWASP Wordpress Security Implementation Guideline](https://www.owasp.org/index.php/OWASP_Wordpress_Security_Implementation_Guideline#Remove_or_change_the_default_administrator_account)

[https://www.owasp.org/index.php/OWASP\\_Wordpress\\_Security\\_Implementation\\_Guideline#Remove\\_or\\_change\\_the\\_default\\_administrator\\_account](https://www.owasp.org/index.php/OWASP_Wordpress_Security_Implementation_Guideline#Remove_or_change_the_default_administrator_account)

[Your WordPress Installation Is Using the Default Admin Account](https://www.acunetix.com/blog/wordpress-security/wordpress-default-admin-account/)

<https://www.acunetix.com/blog/wordpress-security/wordpress-default-admin-account/>

[Change WordPress admin username for security](https://www.inmotionhosting.com/support/website/wordpress/change-wordpress-admin-username-for-security)

<https://www.inmotionhosting.com/support/website/wordpress/change-wordpress-admin-username-for-security>

# WordPress Improper Neutralization of Script-Related HTML Tags in a Web Page (Basic XSS) Vulnerability

In affected versions of WordPress, when uploading themes, the name of the theme folder can be crafted in a way that could lead to JavaScript execution in /wp-admin on the themes page. This does require an admin to upload the theme, and is low severity self-XSS. This has been patched in version 5.4.2, along with all the previously affected versions via a minor release (5.3.4, 5.2.7, 5.1.6, 5.0.10, 4.9.15, 4.8.14, 4.7.18, 4.6.19, 4.5.22, 4.4.23, 4.3.24, 4.2.28, 4.1.31, 4.0.31, 3.9.32, 3.8.34, 3.7.34).

## Impact

<http://192.168.0.103/>

wordpress v4.7.10-4.7.10

## References

[CVE-2020-4049](https://nvd.nist.gov/vuln/detail/CVE-2020-4049)

<https://nvd.nist.gov/vuln/detail/CVE-2020-4049>

# WordPress REST API User Enumeration

WordPress includes a REST API that can be used to list the information about the registered users on a WordPress installation. The REST API exposed user data for all users who had authored a post of a public post type. WordPress 4.7.1 limits this to only post types which have specified that they should be shown within the REST API.

## Impact

An unauthenticated attacker can gain access to the list of users on a WordPress installation. This can be exploited by bots that are launching brute-force password guessing attacks on WordPress websites.

<http://192.168.0.103/>

## Request

```
GET /?rest_route=/wp/v2/users HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## Recommendation

Install a WordPress plugin such as Stop User Enumeration. Stop User Enumeration is a security plugin designed to detect and prevent hackers scanning your site for user names.

## References

### [Stop User Enumeration](https://wordpress.org/plugins/stop-user-enumeration/)

<https://wordpress.org/plugins/stop-user-enumeration/>

### [WordPress 4.7.1 Security and Maintenance Release](https://wordpress.org/news/2017/01/wordpress-4-7-1-security-and-maintenance-release/)

<https://wordpress.org/news/2017/01/wordpress-4-7-1-security-and-maintenance-release/>

# Content Security Policy (CSP) Not Implemented

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.

Content Security Policy (CSP) can be implemented by adding a **Content-Security-Policy** header. The value of this header is a string containing the policy directives describing your Content Security Policy. To implement CSP, you should define lists of allowed origins for the all of the types of resources that your site utilizes. For example, if you have a simple site that needs to load scripts, stylesheets, and images hosted locally, as well as from the jQuery library from their CDN, the CSP header could look like the following:

```
Content-Security-Policy:
default-src 'self';
script-src 'self' https://code.jquery.com;
```

It was detected that your web application doesn't implement Content Security Policy (CSP) as the CSP header is missing from the response. It's recommended to implement Content Security Policy (CSP) into your web application.

## Impact

CSP can be used to prevent and/or mitigate attacks that involve content/code injection, such as cross-site scripting/XSS attacks, attacks that require embedding a malicious resource, attacks that involve malicious use of iframes, such as clickjacking attacks, and others.

<http://192.168.0.103/>

Paths without CSP header:

- <http://192.168.0.103/readme.html>
- <http://192.168.0.103/wp-content/>
- <http://192.168.0.103/wp-includes/>
- <http://192.168.0.103/wp-admin/images/>
- <http://192.168.0.103/wp-login.php>
- <http://192.168.0.103/wp-content/plugins/akismet/>

- <http://192.168.0.103/index.php>
- <http://192.168.0.103/wp-includes/ID3/>
- <http://192.168.0.103/wp-includes/ID3/getid3.lib.php>
- <http://192.168.0.103/wp-content/languages/>
- <http://192.168.0.103/wp-admin/install.php>
- <http://192.168.0.103/wp-includes/IXR/>
- <http://192.168.0.103/wp-includes/IXR/class-IXR-base64.php>
- <http://192.168.0.103/wp-content/themes/>
- <http://192.168.0.103/wp-admin/upgrade.php>
- <http://192.168.0.103/wp-includes/Requests/>
- <http://192.168.0.103/wp-admin/includes/>
- <http://192.168.0.103/wp-includes/SimplePie/>
- <http://192.168.0.103/wp-includes/Text/>
- <http://192.168.0.103/wp-admin/js/>
- <http://192.168.0.103/wp-includes/Text/Diff.php>

## Request

---

GET /readme.html HTTP/1.1  
Referer: http://192.168.0.103/  
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,\*/\*;q=0.8  
Accept-Encoding: gzip,deflate,br  
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36  
Host: 192.168.0.103  
Connection: Keep-alive

## Recommendation

---

It's recommended to implement Content Security Policy (CSP) into your web application. Configuring Content Security Policy involves adding the **Content-Security-Policy** HTTP header to a web page and giving it values to control resources the user agent is allowed to load for that page.

## References

---

### [Content Security Policy \(CSP\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP)

<https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP>

### [Implementing Content Security Policy](https://hacks.mozilla.org/2016/02/implementing-content-security-policy/)

<https://hacks.mozilla.org/2016/02/implementing-content-security-policy/>

# Error page web server version disclosure

---

Application errors or warning messages may disclose sensitive information about an application's internal workings to an attacker.

Acunetix found the web server version number and a list of modules enabled on the target server. Consult the 'Attack details' section for more information about the affected page.

## Impact

---

Error messages information about an application's internal workings may be used to escalate attacks.

<http://192.168.0.103/>

## Request

```
GET /GfK9oYBd5K HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## Recommendation

Properly configure the web server not to disclose information about an application's internal workings to the user. Consult the 'Web references' section for more information.

## References

[Custom Error Responses \(Apache HTTP Server\)](https://httpd.apache.org/docs/current/custom-error.html)

<https://httpd.apache.org/docs/current/custom-error.html>

[server\\_tokens \(Nginx\)](http://nginx.org/en/docs/http/nginx_core_module.html#server_tokens)

[http://nginx.org/en/docs/http/nginx\\_core\\_module.html#server\\_tokens](http://nginx.org/en/docs/http/nginx_core_module.html#server_tokens)

[Remove Unwanted HTTP Response Headers \(Microsoft IIS\)](https://blogs.msdn.microsoft.com/varunm/2013/04/23/remove-unwanted-http-response-headers/)

<https://blogs.msdn.microsoft.com/varunm/2013/04/23/remove-unwanted-http-response-headers/>

# Generic Email Address Disclosure

One or more email addresses have been found on this website. The majority of spam comes from email addresses harvested off the internet. The spam-bots (also known as email harvesters and email extractors) are programs that scour the internet looking for email addresses on any website they come across. Spambot programs look for strings like myname@mydomain.com and then record any addresses found.

## Impact

Email addresses posted on Web sites may attract spam.

<http://192.168.0.103/>

Emails found:

- <http://192.168.0.103/license.txt>  
[m@tidakada.com](mailto:m@tidakada.com)
- <http://192.168.0.103/wp-includes/ID3/readme.txt>  
[mathewhendry@hotmail.com](mailto:mathewhendry@hotmail.com)

## Request

```
GET /license.txt HTTP/1.1
Referer: http://192.168.0.103/
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## Recommendation

Check references for details on how to solve this problem.

## References

[Anti-spam techniques](#)

[https://en.wikipedia.org/wiki/Anti-spam\\_techniques](https://en.wikipedia.org/wiki/Anti-spam_techniques)

# Permissions-Policy header not implemented

The Permissions-Policy header allows developers to selectively enable and disable use of various browser features and APIs.

## Impact

### <http://192.168.0.103/>

Locations without Permissions-Policy header:

- <http://192.168.0.103/readme.html>
- <http://192.168.0.103/wp-content/>
- <http://192.168.0.103/wp-includes/>
- <http://192.168.0.103/wp-admin/images/>
- <http://192.168.0.103/wp-login.php>
- <http://192.168.0.103/wp-content/plugins/akismet/>
- <http://192.168.0.103/index.php>
- <http://192.168.0.103/icons/>
- <http://192.168.0.103/wp-includes/ID3/>
- <http://192.168.0.103/wp-includes/ID3/getid3.lib.php>
- <http://192.168.0.103/wp-content/languages/>
- <http://192.168.0.103/wp-admin/install.php>
- <http://192.168.0.103/wp-includes/IXR/>
- <http://192.168.0.103/wp-includes/IXR/class-IXR-base64.php>
- <http://192.168.0.103/wp-content/themes/>
- <http://192.168.0.103/wp-admin/upgrade.php>
- <http://192.168.0.103/wp-includes/Requests/>
- <http://192.168.0.103/wp-admin/includes/>
- <http://192.168.0.103/wp-includes/SimplePie/>
- <http://192.168.0.103/wp-includes/Text/>
- <http://192.168.0.103/wp-admin/js/>

## Request

```
GET /readme.html HTTP/1.1
Referer: http://192.168.0.103/
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## References

[Permissions-Policy / Feature-Policy \(MDN\)](#)

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Feature-Policy>

[Permissions Policy \(W3C\)](#)

<https://www.w3.org/TR/permissions-policy-1/>

# Subresource Integrity (SRI) Not Implemented

Subresource Integrity (SRI) is a security feature that enables browsers to verify that third-party resources they fetch (for example, from a CDN) are delivered without unexpected manipulation. It works by allowing developers to provide a cryptographic hash that a fetched file must match.

Third-party resources (such as scripts and stylesheets) can be manipulated. An attacker that has access or has hacked the hosting CDN can manipulate or replace the files. SRI allows developers to specify a base64-encoded cryptographic hash of the resource to be loaded. The integrity attribute containing the hash is then added to the `<script>` HTML element tag. The integrity string consists of a base64-encoded hash, followed by a prefix that depends on the hash algorithm. This prefix can either be sha256, sha384 or sha512.

The script loaded from the external URL specified in the Details section doesn't implement Subresource Integrity (SRI). It's recommended to implement Subresource Integrity (SRI) for all the scripts loaded from external hosts.

## Impact

An attacker that has access or has hacked the hosting CDN can manipulate or replace the files.

### <http://192.168.0.103/index.php>

Pages where SRI is not implemented:

- <http://192.168.0.103/index.php>  
Script SRC: <http://dc-2/wp-includes/js/jquery/jquery.js?ver=1.12.4>

## Request

```
GET /index.php HTTP/1.1
Referer: http://192.168.0.103/
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip,deflate,br
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/125.0.0.0 Safari/537.36
Host: 192.168.0.103
Connection: Keep-alive
```

## Recommendation

Use the SRI Hash Generator link (from the References section) to generate a `<script>` element that implements Subresource Integrity (SRI).

For example, you can use the following `<script>` element to tell a browser that before executing the <https://example.com/example-framework.js> script, the browser must first compare the script to the expected hash, and verify that there's a match.

```
<script src="https://example.com/example-framework.js"
integrity="sha384-oqVuAfXRKap7fdgcCY5uykM6+R9GqQ8K/uxy9rx7HNQlGYl1kPzQh01wx4JwY8wC"
crossorigin="anonymous"></script>
```

## References

### [Subresource Integrity](https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity)

[https://developer.mozilla.org/en-US/docs/Web/Security/Subresource\\_Integrity](https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity)

### [SRI Hash Generator](https://www.srihash.org/)

<https://www.srihash.org/>

Coverage

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>http://192.168.0.103</div> <div><div>Inputs</div><div>GETrest_route</div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <div>"%1\$s</div> <div>"</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <div>"%2\$s</div> <div>"</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <div>"%3\$s</div> <div>"</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <div>"%s</div> <div>"</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <div>"https:"</div> <div><div>codex.wordpress.org</div><div><div>Create_A_Network</div><div>"</div></div><div><div>Network_Admin_Settings_Screen</div><div>"</div></div><div><div>Network_Admin_Sites_Screen</div><div>"</div></div><div><div>Network_Admin_Themes_Screen</div><div>"</div></div><div><div>Network_Admin_Updates_Screen</div><div>"</div></div><div><div>Network_Admin_Users_Screen</div><div>"</div></div><div><div>Network_Admin</div><div>"</div></div><div><div>Tools_Network_Screen</div><div>"</div></div></div> <div><div>wordpress.org</div><div><div>support</div><div><div>forum</div><div><div>multisite</div><div>"</div></div></div></div></div> |
| <div>icons</div>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <div><div>wp-admin</div><div><div>css</div><div>install.css</div></div><div>images</div><div>includes</div><div>js</div><div>user</div><div>import.php</div><div>install.php</div><div>update-core.php</div><div>upgrade.php</div></div>                                                                                                                                                                                                                                                                                                                                                                                                                                      |

|                               |
|-------------------------------|
| wp-content                    |
| languages                     |
| plugins                       |
| themes                        |
| %1\$s                         |
| twentyfifteen-en_AU.mo        |
| twentyfifteen-en_AU.po        |
| twentyseventeen-en_AU.mo      |
| twentyseventeen-en_AU.po      |
| twentysixteen-en_AU.mo        |
| twentysixteen-en_AU.po        |
| %1\$s                         |
| %2\$s                         |
| %3\$s                         |
| %s                            |
| admin-en_AU.mo                |
| admin-en_AU.po                |
| admin-network-en_AU.mo        |
| admin-network-en_AU.po        |
| en_AU.mo                      |
| en_AU.po                      |
| plugins                       |
| akismet                       |
| readme.txt                    |
| themes                        |
| wp-includes                   |
| certificates                  |
| ca-bundle.crt                 |
| css                           |
| admin-bar-rtl.css             |
| admin-bar-rtl.min.css         |
| admin-bar.css                 |
| admin-bar.min.css             |
| buttons-rtl.css               |
| buttons-rtl.min.css           |
| buttons.css                   |
| buttons.min.css               |
| customize-preview-rtl.css     |
| customize-preview-rtl.min.css |
| customize-preview.css         |
| customize-preview.min.css     |
| dashicons.css                 |
| dashicons.min.css             |
| editor-rtl.css                |
| editor-rtl.min.css            |
| editor.css                    |
| editor.min.css                |
| jquery-ui-dialog-rtl.css      |

 jquery-ui-dialog-rtl.min.css

 jquery-ui-dialog.css

 jquery-ui-dialog.min.css

 media-views-rtl.css

 media-views-rtl.min.css

 media-views.css

 media-views.min.css

 wp-auth-check-rtl.css

 wp-auth-check-rtl.min.css

 wp-auth-check.css

 wp-auth-check.min.css

 wp-embed-template-ie.css

 wp-embed-template-ie.min.css

 wp-embed-template.css

 wp-embed-template.min.css

 wp-pointer-rtl.css

 wp-pointer-rtl.min.css

 wp-pointer.css

 wp-pointer.min.css

 customize

 fonts

 ID3

 getid3.lib.php

 getid3.php

 license.commercial.txt

 license.txt

 module.audio-video.asf.php

 module.audio-video.flv.php

 module.audio-video.matroska.php

 module.audio-video.quicktime.php

 module.audio-video.riff.php

 module.audio.ac3.php

 module.audio.dts.php

 module.audio.flac.php

 module.audio.mp3.php

 module.audio.ogg.php

 module.tag.apetag.php

 module.tag.id3v1.php

 module.tag.id3v2.php

 module.tag.lyrics3.php

 readme.txt

 images

 IXR

 class-IXR-base64.php

 class-IXR-client.php

 class-IXR-clientmulticall.php

 class-IXR-date.php

|                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------|
|  class-IXR-error.php                     |
|  class-IXR-introspectionserver.php      |
|  class-IXR-message.php                 |
|  class-IXR-request.php                 |
|  class-IXR-server.php                  |
|  class-IXR-value.php                   |
|  js                                    |
|  pomo                                  |
|  random_compat                         |
|  Requests                              |
|  rest-api                              |
|  SimplePie                             |
|  Text                                  |
|  Diff                                  |
|  Diff.php                              |
|  theme-compat                          |
|  widgets                               |
|  class-wp-nav-menu-widget.php          |
|  class-wp-widget-archives.php          |
|  class-wp-widget-calendar.php          |
|  class-wp-widget-categories.php        |
|  class-wp-widget-links.php             |
|  class-wp-widget-meta.php            |
|  class-wp-widget-pages.php           |
|  class-wp-widget-recent-comments.php |
|  class-wp-widget-recent-posts.php    |
|  class-wp-widget-rss.php             |
|  class-wp-widget-search.php          |
|  class-wp-widget-tag-cloud.php       |
|  class-wp-widget-text.php            |
|  admin-bar.php                       |
|  atomlib.php                         |
|  author-template.php                 |
|  bookmark-template.php               |
|  bookmark.php                        |
|  cache.php                           |
|  canonical.php                       |
|  capabilities.php                    |
|  category-template.php               |
|  category.php                        |
|  class-feed.php                      |
|  class-http.php                      |
|  class-IXR.php                       |
|  class-json.php                      |
|  class-oembed.php                    |
|  class-phpass.php                    |
|  class-phpmailer.php                 |

 class-pop3.php

 class-requests.php

 class-simplepie.php

 class-smtp.php

 class-snoopy.php

 class-walker-category-dropdown.php

 class-walker-category.php

 class-walker-comment.php

 class-walker-nav-menu.php

 class-walker-page-dropdown.php

 class-walker-page.php

 class-wp-admin-bar.php

 class-wp-ajax-response.php

 class-wp-comment-query.php

 class-wp-comment.php

 class-wp-customize-control.php

 class-wp-customize-manager.php

 class-wp-customize-nav-menus.php

 class-wp-customize-panel.php

 class-wp-customize-section.php

 class-wp-customize-setting.php

 class-wp-customize-widgets.php

 class-wp-dependency.php

 class-wp-editor.php

 class-wp-embed.php

 class-wp-error.php

 class-wp-feed-cache-transient.php

 class-wp-feed-cache.php

 class-wp-hook.php

 class-wp-http-cookie.php

 class-wp-http-curl.php

 class-wp-http-encoding.php

 class-wp-http-ixr-client.php

 class-wp-http-proxy.php

 class-wp-http-requests-hooks.php

 class-wp-http-requests-response.php

 class-wp-http-response.php

 class-wp-http-streams.php

 class-wp-image-editor-gd.php

 class-wp-image-editor-imagick.php

 class-wp-image-editor.php

 class-wp-list-util.php

 class-wp-locale-switcher.php

 class-wp-locale.php

 class-wp-matchesmapregex.php

 class-wp-meta-query.php

 class-wp-metadata-lazyloader.php

 class-wp-network-query.php

 class-wp-network.php

 class-wp-oembed-controller.php

 class-wp-post-type.php

 class-wp-post.php

 class-wp-query.php

 class-wp-rewrite.php

 class-wp-role.php

 class-wp-roles.php

 class-wp-session-tokens.php

 class-wp-simplepie-file.php

 class-wp-simplepie-sanitize-kses.php

 class-wp-site-query.php

 class-wp-site.php

 class-wp-tax-query.php

 class-wp-taxonomy.php

 class-wp-term-query.php

 class-wp-term.php

 class-wp-text-diff-renderer-inline.php

 class-wp-text-diff-renderer-table.php

 class-wp-theme.php

 class-wp-user-meta-session-tokens.php

 class-wp-user-query.php

 class-wp-user.php

 class-wp-walker.php

 class-wp-widget-factory.php

 class-wp-widget.php

 class-wp-xmlrpc-server.php

 class-wp.php

 class.wp-dependencies.php

 class.wp-scripts.php

 class.wp-styles.php

 comment-template.php

 comment.php

 compat.php

 cron.php

 date.php

 default-constants.php

 default-filters.php

 default-widgets.php

 deprecated.php

 embed-template.php

 embed.php

 feed-atom-comments.php

 feed-atom.php

 feed-rdf.php

 feed-rss.php

|                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------|
|  feed-rss2-comments.php          |
|  feed-rss2.php                  |
|  feed.php                      |
|  formatting.php                |
|  functions.php                 |
|  functions.wp-scripts.php      |
|  functions.wp-styles.php       |
|  general-template.php          |
|  http.php                      |
|  kses.php                      |
|  l10n.php                      |
|  link-template.php             |
|  load.php                      |
|  locale.php                    |
|  media-template.php            |
|  media.php                     |
|  meta.php                      |
|  ms-blogs.php                  |
|  ms-default-constants.php      |
|  ms-default-filters.php        |
|  ms-deprecated.php             |
|  ms-files.php                 |
|  ms-functions.php            |
|  ms-load.php                 |
|  ms-settings.php             |
|  nav-menu-template.php       |
|  nav-menu.php                |
|  option.php                  |
|  pluggable-deprecated.php    |
|  pluggable.php               |
|  plugin.php                  |
|  post-formats.php            |
|  post-template.php           |
|  post-thumbnail-template.php |
|  post.php                    |
|  query.php                   |
|  registration-functions.php  |
|  registration.php            |
|  rest-api.php                |
|  index.php                   |
|  #fragments                  |
|  content                     |
|  license.txt                 |
|  readme.html                 |
|  wp-login.php                |
|  Inputs                      |
| <input type="POST"/> log, pwd, wp-submit                                                                        |

